BS COMPUTER SCIENCE · FINAL YEAR PROJECT

Plotline — full implementation record

End to end: what was built, which code does it, how that code runs, and the result of each function. Checked against the running API on 12 November 2026.

Plotline measures a plot, converts the area, matches a catalogue house with rules, draws that house in metres, and answers planning questions from the stored fields. The language model does not measure, does not pick the house, and does not invent a tax. This document walks every stage in the order a person uses the product.

1. Product path

```
Register or log in
  -> Measure: AI Camera  OR  length x width  OR  WebXR corners
  -> Area in m2 (shoelace or rectangle) -> Marla / Kanal / Acre
  -> Pick a style -> catalogue match on min/max m2
  -> Feasibility: footprint, leftover plot, coverage %
  -> 3D massing (day or night) and optional camera preview
  -> Report and Ask, with or without saving a project
```

The browser is Next.js (React, TypeScript, Three.js). The API is Django REST Framework on SQLite. On this computer the API is port 8000 and the site is port 3000. A phone on Netlify does not call localhost. It calls NEXT_PUBLIC_API_ORIGIN, which is an HTTPS tunnel to that same Django process. resolveApiUrl in frontend/lib/api.ts prefixes /api and /media with that origin when it is set, and leaves the path relative when the Next.js dev server is proxying Django.

```
export function resolveApiUrl(path: string) {
  const normalized = normalizeApiPath(path);
  const origin = (process.env.NEXT_PUBLIC_API_ORIGIN || "").replace(/\/$/, "");
  return origin ? `${origin}${normalized}` : normalized;
}
```

Result tonight. GET http://127.0.0.1:8000/api/health/ returned HTTP 200: ok true, llm_configured true, model Qwen/Qwen2.5-72B-Instruct, depth model depth-anything/Depth-Anything-V2-Small-hf, type relative. The disclaimer in that JSON says measurements are planning estimates, not a land survey.

2. Accounts

RegisterView and LoginView live in backend/planning/views.py and are mounted at /api/auth/register/ and /api/auth/login/. Login accepts a username or an email. resolve_login_user tries authenticate() on the raw string, then looks up User by email and authenticates the stored username. Success returns a DRF token. The browser keeps it in localStorage under plotline.token and sends Authorization: Token on later calls. Logout deletes that token. Saving a project and regenerating its stored report require a logged-in user. The preview report and preview Ask do not.

Result. An empty preview Ask (no question) is rejected with HTTP 400 and the API log line “question is required”. That check is the gate before any consultant text is built.

3. Area from corners or from length and width

World points are y-up. Land area uses only X and Z. Height is dropped because the plot is treated as flat. The same rules exist in frontend/lib/geometry/area.ts and are mirrored on the server so a saved project cannot disagree with the screen.

```
export function areaSqmFromPoints(points) {
  return assessPolygon(points).areaSqm;
}
// 2 points -> need_third_corner, area 0. Length x width is a separate entry.
// crossed corners -> self_intersecting, area 0.
// shoelace under 0.05 m2 -> collinear, area 0. Nothing is squared from a span.
function shoelaceXZ(points) {
  acc += x_i * z_{i+1} - x_{i+1} * z_i
  return abs(acc) / 2
}
```

Manual entry is the same rectangle: area = length times width. A 15 m by 12 m plot is 180 m². The tape-length field on the camera screen starts empty. Leaving it empty blocks a metric area, so the program cannot invent a scale.

3.1 Taps on the photograph

displayToImageCoords in frontend/lib/geometry/displayToImage.ts converts a click on the on-screen video into a pixel of the native photograph. The video is drawn with object-fit contain, so there can be letterbox bars. A click in the bar returns null and is ignored. A click on the picture is scaled by the contain fit, not by CSS width alone and not by devicePixelRatio. That is what keeps the polygon on the same ground the depth model saw.

Result. The formula is deterministic. Two corners at (0,0) and (15,12) give 180 m². Four corners of that rectangle give the same 180 m² through shoelace. The UI will not produce metres until a known length is entered on the camera path.

4. AI Camera — one photograph, not SLAM

AIMeasurement.tsx opens getUserMedia and shows the preview locally. Nothing is uploaded until Analyze Scene. That button posts one JPEG to POST /api/vision/depth/. Django (views_vision.py and vision_measurement.py) runs OpenCV — resize, denoise, CLAHE, edges — then Depth Anything V2 Small, an Apache-2.0 relative depth model of about 24.8 million parameters, on the machine. SegFormer, via POST /api/vision/segment/, marks ground against obstacles. A tree or a person inside the polygon is rejected. The person then taps 3 to 8 corners and types the real tape distance between two of them. POST /api/vision/measure/ unprojects those pixels onto the plane, scales the relative depth by that tape length, and calls the shoelace code. The JPEG is not stored and is not sent to the consultant.

```
Phone preview (no upload)
  Analyze Scene -> one JPEG -> /api/vision/depth/
  OpenCV + Depth Anything V2 Small + ground RANSAC
  /api/vision/segment/  -> ground vs obstacle
  taps + known length (metres) -> /api/vision/measure/
  unproject -> scale -> shoelace -> m2 -> planning
```

WebXR remains a third path. On Android Chrome with immersive-ar, the existing hit-test stores world points and the same area function runs. Anchors and the hit-test reticle were not rewritten when the camera preview was added. iOS Safari usually cannot open immersive-ar, so those phones use section 10.

Result. The vision routes answer. With VISION_MOCK=1 the depth model is not required for a UI run; a real analyze still goes through the same three URLs. The screen shows Measurement Quality and refuses a metric area when the tape field is empty. The product text says this is not a professional survey.

5. Marla, Kanal, and Acre

backend/planning/units.py holds fixed constants, not learned ones. 1 Marla = 272.25 sq ft = 25.29285264 m². 1 Kanal = 20 Marla. 1 Acre = 160 Marla (Punjab, 8 kanal). sqm_to_units divides the measured square metres by those constants and picks a display unit: kanal if the plot is at least one kanal, otherwise marla if it is at least one marla, otherwise square metres.

```
MARLA_SQM = 25.29285264
marla = area / MARLA_SQM
kanal = area / (MARLA_SQM * 20)
acre  = area / (MARLA_SQM * 160)
```

Result, live, for 180 m². sqft 1937.5, marla 7.117, kanal 0.3558, acre 0.04448, display label “7.12 Marla (180.0 m²)”. The same object is returned inside the design-match payload.

6. Matching a house — rules, not a model

recommend_house in backend/planning/recommendation.py loads active HouseDesign rows ordered by recommended_min_plot. It keeps rows whose inclusive range contains the measured area. If several match, it picks the one whose range midpoint is closest to the area. If none match, it still returns the nearest design and the reason says the plot is outside every published range. The reason string is what the result screen shows. There is no training step.

```
in_range = [h for h in catalog
            if h.recommended_min_plot <= size <= h.recommended_max_plot]
chosen = min(in_range, key=lambda h: abs(midpoint(h) - size))
```

The style screen does not call that function blindly. GET /api/designs/match/?style=...&plot;_area=... filters the same ranges inside the chosen style and splits exact matches from nearby ones. Styles in the seed are Modern, Italian Villa, American, Cottage / Hut, Contemporary, Traditional, and Luxury Villa. Fifteen houses are seeded, from Modern Compact to Luxury Villa Premium. English, Minimalist, and Spanish were not added.

Design	Id	Size	Plot band	List / band
Italian Compact Villa	8	10 x 9 x 7 m, 2 floors	120–180 m²	PKR 16,000,000 (14–19M)
Italian Family Villa	9	12 x 11 x 8.5 m, 2 floors	160–260 m²	PKR 24,000,000 (20–28M)
Italian Large Villa	10	16 m wide, 2 floors	above the compact band	PKR 45,000,000

Result, live, style Italian Villa, plot 180 m². match_kind exact. Exact rows: id 8 Italian Compact Villa and id 9 Italian Family Villa. Nearby: id 10 Italian Large Villa. Compact is the design whose range ends on 180, so it is the worked example below. model_type is procedural. source_license is project-generated. The attribution string says it is generated architectural massing, not a photoreal scan.

7. Feasibility

backend/planning/feasibility.py does not call a model. Footprint is the stored building_footprint_sqm, or width times depth if that field is empty. Remaining area is plot minus footprint. Coverage is footprint divided by plot, times 100. If the footprint is larger than the plot the status says so. If coverage is above 70 percent the note says coverage is high. Otherwise the status is suitable_preliminary. A separate space band (bedrooms, baths, parking) is chosen from the plot size alone, for the style screen’s rough programme. It does not override the room counts stored on the chosen house.

```
footprint = building_footprint_sqm or (width * depth)
remaining = plot - footprint
coverage  = footprint / plot * 100
```

Result, live, POST /api/feasibility/ with land_size_sqm 180 and house_id 8. Footprint 90.0 m², remaining 90.0 m², ground coverage 50.0 percent, status suitable_preliminary, note “Footprint fits within plot area for preliminary visualization.” The disclaimer says this is not a bylaw check, a structural design, or a municipal approval. Room programme on that house: 3 bedrooms, 3 bathrooms, 1 powder, 1 living, 1 family, 1 dining, 1 drawing, 1 kitchen,

parking 1 (max 2), 2 balconies, 1 terrace, garden yes, garage no.

8. Cost, LDA, and DHA

The construction number is the catalogue band on the house row (estimated_cost_min, estimated_cost_pkr, estimated_cost_max). The screen labels it a preliminary estimate. The consultant in backend/planning/services/local_rag.py can also mention a published residential map-fee illustration of PKR 15 per square foot. That rate is a reference (the reported LDA residential revision, and the June 2024 note of the MCL gazette move to Rs 15). It is not read from a challan API.

detect_intent walks a fixed list and returns on the first hit. Authority patterns (DHA, LDA versus DHA, “how do I know”) are tested before fee patterns. Fee patterns include the word lda, tax, challan, stamp duty. So “is this DHA or LDA?” does not become the fee essay, and “add the LDA tax” to a budget does not become a beauty tip. The word “add” used to match a beauty sentence; it is now a stop-word unless a fee or budget intent wins first.

```
def detect_intent(question):
    for intent, patterns in _INTENT_PATTERNS:  # authority, then fees, then build, cost, ...
        for p in patterns:
            if re.search(p, question.lower()):
                return intent
    return "general"
```

If the question both asks to add or include a fee and talks about budget or construction, the answer is a combined sheet: catalogue band plus PKR 15/sq ft. Otherwise a bare “is tax included?” is answered “No”, and the fee is shown only as an illustration that was excluded. Hugging Face is not allowed to rewrite fee, authority, or build answers, so it cannot replace them with style tips.

8.1 Live answers for Italian Compact Villa, 180 m²

Question: “is in that we add the LDA tax?” Source local_rag, intent fees. Answer begins: “No. LDA tax is not included in the cost for Italian Compact Villa.” It states the band PKR 14,000,000–19,000,000 and lists what is outside it: LDA or MCL map fee, DHA / RUDA / cantonment charges, stamp duty, and utility connections.

Question: “give me the construction budget and add the LDA tax.” The construction range in that answer is only the catalogue band stored on the design. A second Lahore rupee-per-square-foot range is not printed. LDA map-approval stays a fee illustration on top of that one band. The seed prices are catalogue assumptions, not a quotation. The file is not recorded as DHA or LDA, and the total is not a challan.

Question: “how do I know if this land is DHA or LDA?” Source local_rag, intent authority. First sentence: “This project does not record whether the land is DHA or LDA.” It then repeats the stored house facts and tells the person to use the allotment, transfer, or registry. It does not repeat the fee essay as the answer.

9. The 3D house

Rendering is Three.js through React Three Fiber only. HouseRenderer in frontend/components/architecture/ArchViewer.tsx checks model_type. Procedural designs, which is the whole seeded catalogue, draw ArchitecturalHouse. A GLB path still exists and uses the Draco decoder under public/draco, but the label stays “Generated massing model”. Modern, Contemporary, and Luxury delegate to ModernReferenceHouse. Italian, American, Cottage, and Traditional stay in ArchitecturalHouse: a mass in metres, origin at the footprint centre, front toward +Z, storey height 3.15 m, with columns, arches, a pitched roof, and a site pad.

The result screen (ResultScreen.tsx) is a sheet. On a phone the snaps are peek, half, and full. The canvas height is the viewport minus the sheet, because a camera view-offset kept the roof and hid the facade. Desktop is a 65/35 split. Tabs are Overview, Rooms, Feasibility, Report, and Ask. Day and Evening change exposure, sky, glass tint, and window glow. Report and Ask call POST /api/preview/report/ and POST /api/preview/ask/ with house_id and plot_area_sqm, so they work before a project is saved. Saving still creates a Project row for later.

9.1 Surfaces started for a more real finish

frontend/components/architecture/surfaceTextures.ts draws original textures on a canvas at runtime. plasterMaps speckles a stucco colour and builds a bump map. roofTileMap draws overlapping tile rows for pitched roofs. grassMap draws short blades. stonePaveMap draws jointed flags. timberMap draws grain. useArchMaterials assigns those maps to wall, roof, grass, and paving for the Italian and the other non-modern styles. ModernReferenceHouse uses the plaster, grass, stone, and timber maps on its own materials. Material colour is white where the map already carries the colour, so the map is not tinted twice. These are not photographs and not a scan. Cone trees, simple mullions, and a closed interior remain. Blender was not connected, so no authored hero mesh was exported.

Earlier visual check, headless Chromium, not a phone: about 60 frames per second for two seconds at 390×844 with the CPU slowed four times, first result paint about 3.7 seconds, no console errors on that run. Immersive AR was not available in that browser, so anchor placement was not re-measured on a device.

10. Camera preview

If the browser cannot open immersive-ar, CameraPreview.tsx shows the environment camera and the same HouseRenderer at 1:1 scale. The gyro may rotate the view. Walking is not tracked. On a laptop the picture is the webcam, and a floor tap often misses because the lens is looking at a person. The house is therefore placed as soon as the preview opens.

An 8 m stand-off put the camera inside a 12 m villa, so a 90° field of view still showed one wall. fitStandOff in frontend/lib/ar/previewFov.ts computes how far back the centre must sit so width, height, and the front face fit, with a 1.35 margin, then clamps the result between 10 m and 56 m. The Zoom out slider moves that distance. Reset frames the house again. The idle camera looks at the middle of the facade. When a real device orientation arrives, the gyro takes over.

```
distW = (width/2 * 1.35) / tan(hFov/2)
distH = (height/2 * 1.35) / tan(vFov/2)
front = depth/2 + 2.5
standOff = clamp(max(distW, distH) + front, 10, 56)
# 12 x 11 x 8.5 m at 60 degrees -> about 23 m
```

The lens itself uses visiblePerspective. object-fit cover crops the sensor. The horizontal field of view is the field of the visible crop, in native pixels, not CSS pixels. A check of that formula: 1 m at 2 m subtends about 28°. A 1920×1080 frame covered into 390×844 leaves about 499×1080 native pixels, and 60° horizontal becomes about 103° vertical. Day and Night do not follow the brightness of the webcam. Day is a sun plus a front fill so the portico stays readable in a dark room. Night lowers that sun and turns on the warm window and porch lights inside the massing model. The same evening flag is shared with the result screen.

AlvaAR (GPLv3) was not loaded. Tracking was not shown to hold on a real phone over HTTPS, so the Experimental control only explains that, and the credits page says the same. WebXR session start, anchors, hit-test, measurement, and the recommender were not rewritten for this preview.

11. Scrolling on a phone

The class ar-lock used to set touch-action: none and overflow: hidden on the whole planner so the canvas would not pan the page. On iOS that also froze every panel, because a touch is allowed only if every ancestor allows it. The lock is now overflow and height only. touch-action: none sits on the canvas. Panels use ar-scroll (touch-action: pan-y, overflow-y: auto). The result sheet scrolls its body under the drag handle. Checked at 390×560: Calculate Area could be scrolled fully into view. The credits page, taller than the screen, scrolled as a normal document.

12. Netlify and logs

netlify.toml builds the frontend directory and bakes NEXT_PUBLIC_API_ORIGIN. That value is a Cloudflare quick-tunnel URL and changes when the tunnel process restarts. The tunnel that was checked from this computer reached Django /api/health/ with HTTP 200. If the phone reports that it cannot reach the API and the Django console shows no new line, the request never arrived.

ApiRequestLogMiddleware in backend/planning/middleware.py logs every path under /api/ after the view returns: method, path, status, milliseconds, username if logged in, and a short error or detail on 400 and above. Bodies, passwords, and tokens are not written. PlanningConfig.ready logs debug, vision_mock, the depth model id, and whether a language-model token exists. The token value is not printed. Vision keeps its own lines under the logger planning.vision.

```
api GET /api/health/ -> 200
api POST /api/preview/ask/ -> 400  question is required
api POST /api/preview/ask/ -> 200  (the three answers in section 8)
```

13. Result of each function

Function	Result on 12 November 2026
API health	HTTP 200, ok, LLM configured, depth model named
180 m² units	7.12 Marla, 1937.5 sq ft, 0.3558 Kanal
Italian match at 180 m²	Exact: Compact (8) and Family (9). Nearby: Large (10)
Feasibility, house 8	90 m² footprint, 50% coverage, suitable_preliminary
“Is LDA included?”	No. Band PKR 14–19 million. Fee is outside it.
“Budget and add LDA”	Band plus PKR 15/sq ft. Combined PKR 14,014,532–19,029,062
“DHA or LDA?”	Not stored. Intent authority, not the fee essay.
Empty Ask	HTTP 400, question is required
Camera stand-off	12×11×8.5 m villa frames at about 23 m
Phone scroll	Calculate Area reached on a 560 px tall viewport
Headless 3D frame, earlier run	About 60 fps, about 3.7 s to first paint, no console errors
Desktop Chrome, 12 Nov 2026, Radeon Pro 560X, 1440x900	Before: 14 fps at pixel ratio 2, 622 draws. After: steady 60 fps at pixel ratio 1.5, 141 draws, once the 256 px HDR is filtered. No phone.
Real Android immersive AR	Not re-run
AlvaAR / Blender / photoreal scan	Not shipped

Files that own the behaviour: area.ts and displayToImage.ts; vision_measurement.py and views_vision.py; units.py; recommendation.py; feasibility.py; local_rag.py; ArchitecturalHouse.tsx, ModernReferenceHouse.tsx, surfaceTextures.ts; ResultScreen.tsx; CameraPreview.tsx and previewFov.ts; middleware.py; urls.py under planning.

What this product will not say. It will not say the area is a survey, the cost is a contractor quote, the PKR 15 figure is your challan, the render is a photograph of a built house, or that walking is tracked in camera preview.